COGS 181A - Homework 2

due Tuesday, January 27 at 11:59PM

Instructions: Answer the questions below, attach your code, and insert figures to create a PDF file; submit your file via Gradescope. You may look up the information on the Internet, but you must write the final homework solutions by yourself.

Late Policy: 5% of the total points will be deducted on the first day past due. Every 10% of the total points will be deducted for every extra day past due.

System Setup: You can install Anaconda to setup the Jupyter Notebook environment. Most packages have been already installed in Anaconda. If some package is not installed, you can use `pip` to install the missing package, that is, just type `pip install PACKAGE_NAME` in the terminal.

Problem 1. (10 points) Conceptual Questions

1.1 Is the following statement true or false? For inputs of two-dimensional features $\mathbf{x} = (x_1, x_2)$, a perceptron classifier cannot solve the XOR problem (Figure 1), but a logistic regression classifier can.

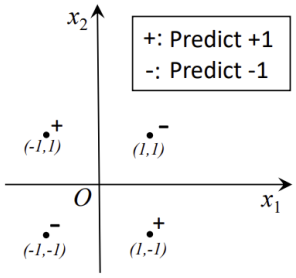


Figure 1: The XOR Problem

> **Solution:** False, a logistic regression classifier cannot solve the XOR problem since it's not linearly separable.

1.2 Choose **all** the valid linear classifiers from the options below:

A. Perceptron.

B. Logistic regression classifier.

C. Multi-layer neural networks classifier.

> **Solution:** A and B

1.3 Choose **all** the valid descriptions about **gradient descent** from the options below:

A. The global minimum can always be reached by using gradient descent.

B. Every gradient descent iteration can always decrease the value of the loss function even when the gradient of the objective function is zero.

C. When the learning rate is very large, some iterations of gradient descent may not decrease the value of the loss function.

D. With different initial weights, the gradient descent algorithm may lead to different local minima.

E. None of the above is valid.

Solution: C and D

1.4 Choose **all** the valid descriptions about the **stochastic gradient descent** algorithm from the options below:

A. Stochastic gradient descent (SGD) always finds the global optimal solution.

B. Stochastic gradient descent (SGD) is typically more computationally efficient than standard gradient descent (GD).

C. Stochastic gradient descent (SGD) is typically more memory-demanding than standard gradient descent (GD).

D. Stochastic gradient descent (SGD) can be used to avoid bad local optima.

E. Stochastic gradient descent (SGD) uses a fixed learning rate.

Solution: B and D

1.5 In this question, we will apply the perceptron learning algorithm to solve a binary classification problem: We need to predict a binary label $y \in \{-1, +1\}$ for a feature vector $\mathbf{x} = [x_0, x_1]^\top$. The decision rule of the perceptron model is defined as:

$$f(\mathbf{x}; \mathbf{w}, b) = \begin{cases} +1, & \text{if } \mathbf{w} \cdot \mathbf{x} + b \geq 0 \\ -1, & \text{otherwise} \end{cases} \tag{1}$$

where $\mathbf{w} = [w_0, w_1]^\top$ is the weight vector, and b is the bias scalar. Given a training dataset $S_{\text{training}} = \{(\mathbf{x}_i, y_i), i = 1, \ldots, n\}$, the outline of the perceptron algorithm is shown below:

- Initialize weight $\mathbf{w}$ and bias b.
- If not all data points in S_{training} are correctly predicted:
  - Obtain the prediction for each feature vector in S_{training}.
  - If the prediction is wrong, then update the parameters $\mathbf{w}$ and b.
- Otherwise return current weight $\mathbf{w}$ and bias b.

In this problem, you will implement a perceptron algorithm. Suppose X is the matrix of all feature vectors $\mathbf{x}_i$ and Y is the matrix of all labels y_i in the training set S_{training}.

Besides, assume a function `calc_error(X, Y, W, b)` is given, which computes the error from the feature matrix X, the label matrix Y, the weight W and the bias b. Please reorder the following lines of Python code to complete your perceptron algorithm. Fill the indexes (a-g) into the blanks.

```
def perceptron_algorithm(X, Y):
    # Initialization.
    W = np.zeros(2)
    b = 0.0
    lam = 1
    # Main algorithm.
    while calc_error(X, Y, W, b) > 0:
    for xi, yi in zip(X, Y):
    ------------------------------------------------
    ----------------------------------------------
    else:
    ----------------------------------------------
    ------------------------------------------------
    ----------------------------------------------
    else:
    ----------------------------------------------
    ----------------------------------------------
    return W, b
    (a) W = W + lam * (yi - yi_pred) * xi
    (b) yi_pred = -1
    (c) b = b + lam * (yi - yi_pred)
    (d) yi_pred = +1
    (e) if W.T.dot(xi) + b >= 0:
    (f) continue
    (g) if yi_pred == yi:
```

Solution: The correct order is e, d, b, g, f, a, c.

Problem 2. (8 points) Perceptron Update Rule

We are given a training set $S = \{(\mathbf{x}_i, y_i), i = 1, 2, \ldots, n\}$, where $\mathbf{x}_i \in \mathbb{R}^m$ and $y_i \in \{-1, +1\}$. Define the objective function for sample i when training the perceptron as:

$$L_i(\mathbf{w}, b) = \max(0, -y_i(\mathbf{w}^\top \mathbf{x}_i + b)).$$

For a chosen sample point i, let $target_i = y_i$ and $output_i = \text{sign}(\mathbf{w}^\top \mathbf{x}_i + b)$. Show your proof that:

2.1 $\frac{\partial L_i(\mathbf{w},b)}{\partial \mathbf{w}} = -\frac{1}{2}(target_i - output_i)\mathbf{x}_i$

Solution: When the label is misclassified, $target_i - output_i = 2y_i$ so $y_i = \frac{1}{2}(target_i - output_i)$. When the label is correctly classified, $target_i - output_i = 0$ so $0 = \frac{1}{2}(target_i - output_i)$.

$$L_i = \max(0, -y_i(\mathbf{w}^\top \mathbf{x}_i + b)) = \begin{cases} 0 & \text{if correctly classified} \\ -y_i(\mathbf{w}^\top \mathbf{x}_i + b) & \text{if misclassified} \end{cases}$$

$$\frac{\partial L_i}{\partial \mathbf{w}} = \begin{cases} 0 & \text{if correctly classified} \\ -y_i \mathbf{x}_i & \text{if misclassified} \end{cases}$$

$$= -\frac{1}{2}(target_i - output_i)\mathbf{x}_i$$

2.2 $\frac{\partial L_i(\mathbf{w},b)}{\partial b} = -\frac{1}{2}(target_i - output_i)$

Solution: Using the same logic as part (a), we get $y_i = \frac{1}{2}(target_i - output_i)$ if misclassified and $0 = \frac{1}{2}(target_i - output_i)$ if correctly classified. Thus,

$$\frac{\partial L_i}{\partial b} = \begin{cases} 0 & \text{if correctly classified} \\ -y_i & \text{if misclassified} \end{cases}$$

$$= -\frac{1}{2}(target_i - output_i)$$

Problem 3. (30 points) Perceptron (Coding)

3.1 (20 points) Perceptron Algorithm - Deterministic Iteration

In this section, we will apply the perceptron learning algorithm to solve the same binary classification problem as logistic regression above: We need to predict a binary label $y \in \{-1, 1\}$ for a feature vector $\mathbf{x} = [x_0, x_1]^\top$. The decision rule of the perceptron model is defined as:

$$f(\mathbf{x}; \mathbf{w}, b) = \begin{cases} +1, & \text{if } \mathbf{w} \cdot \mathbf{x} + b \geq 0 \\ -1, & \text{otherwise} \end{cases} \tag{2}$$

where $\mathbf{w} = [w_0, w_1]^\top$ is the weight vector, and b is the bias scalar. Given a training dataset $S_{\text{training}} = \{(\mathbf{x}_i, y_i), i = 1, \ldots, n\}$, we define the training error e_{training} as:

$$e_{\text{training}} = \frac{1}{n} \sum_{i=1}^{n} \mathbb{I}(y_i \neq f(\mathbf{x}_i; \mathbf{w}, b)) \tag{3}$$

and the test error e_{test} on the test set S_{test} can be defined in the same way. In the perceptron algorithm, we aim to directly minimize the training error e_{training} to obtain the optimal parameters $\mathbf{w}^*, b^*$. If we represent data points in training set S_{training} as matrices $X = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_n]^\top$ and $Y = [y_1, y_2, \ldots, y_n]^\top$, the perceptron algorithm is shown as below:

Algorithm 1 Perceptron Learning Algorithm

Data: Training set S_{training}, which contains feature vectors X and labels Y;
Initialize parameters $\mathbf{w}$ and b; pick a constant $\lambda \in (0, 1]$, which is similar to the step size in the standard gradient descent algorithm ($\lambda = 1$ by default).
while *not every data point is correctly classified* **do**
  for *each feature vector* $\mathbf{x}_i$ *and its label* y_i *in the training set* $S_{training}$ **do**
    compute the model prediction $f(\mathbf{x}_i; \mathbf{w}, b)$;
    if $y_i = f(\mathbf{x}_i; \mathbf{w}, b)$ **then**
      continue;
    else
      update the parameters $\mathbf{w}$ and $\mathbf{b}$:
      $\mathbf{w} \Leftarrow \mathbf{w} + \lambda(y_i - f(\mathbf{x}_i; \mathbf{w}, b))\mathbf{x}_i$
      $\mathbf{b} \Leftarrow \mathbf{b} + \lambda(y_i - f(\mathbf{x}_i; \mathbf{w}, b))$
    end
  end
end

Please fill out the missing blanks in perceptron.ipynb. Follow the instructions in the skeleton code and report:

1. Your code.
2. Equation of decision boundary corresponding to the optimal $\mathbf{w}^*$ and b^*.
3. Plot of training set along with decision boundary.
4. Plot of test set along with decision boundary.
5. Training error and test error.

6. Training error curve.

Hint:

1. Multiplication inside `f_perceptron` is matrix-vector (or vector-vector) multiplication. You can use `dot`.
2. Inside the `update rule`, λ and $y_i - f(\mathbf{x}_i; \mathbf{w}, b)$ are scalar while $\mathbf{x}_i$ is a vector. You should consider an element-wise multiplication using `*`.

```
def indicator(gt_label, pred_label):
    return int(gt_label != pred_label)

def f_perceptron(x, W, b):
    return np.sign(np.dot(x, W) + b)

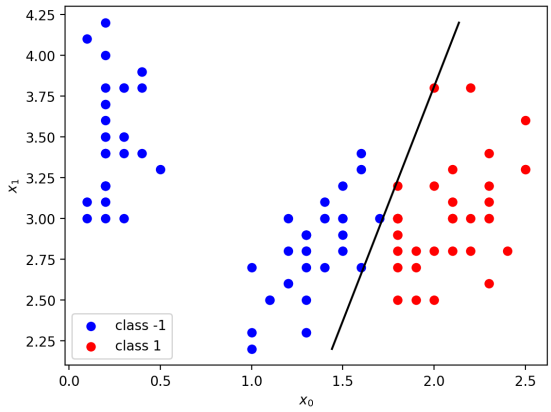
def calc_error(X, Y, W, b):
    errors = np.array([])
    for (xi, yi) in zip(X, Y):
        error = indicator(yi, f_perceptron(xi, W, b))
        errors = np.append(errors, error)
    return np.mean(errors)

# Some settings.
errors = []           # Error history.
lam    = 1            # Lambda which controls the step size.

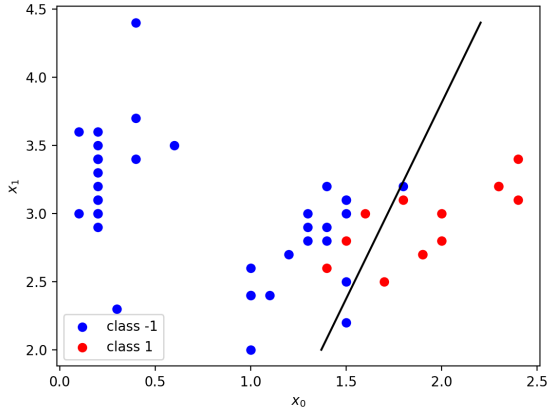
# Initialization.
W      = np.zeros(2)  # Weight.
b      = 0.0          # Bias.

# Perceptron learning algorithm.
while calc_error(X_train, Y_train, W, b) > 0:
    for xi, yi in zip(X_train, Y_train):   # Iterate over all data points.
        y_pred = f_perceptron(xi, W, b) # Compute the model prediction.
        if yi == y_pred:    # Compare prediction and label.
            continue                       # - If correct, continue.
        else:
            W += lam * (yi - y_pred) * xi
            b += lam * (yi - y_pred)
    errors.append(calc_error(X_train, Y_train, W, b))
```

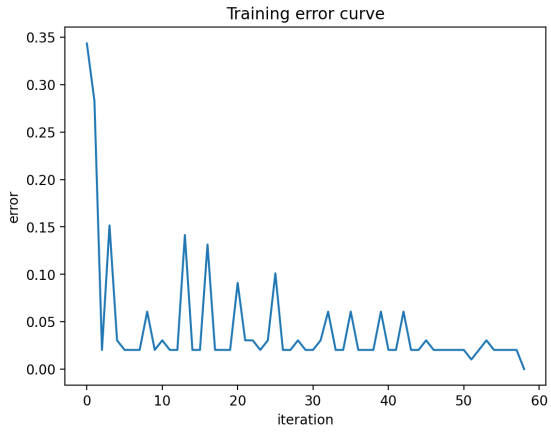
Solution: Decision boundary: $69.600x_0 - 24.200x_1 - 47.000 = 0$.



Training error: 0.0



Test error: 0.1



3.2 (10 points) Perceptron Algorithm - Random Sample

In this section, we want to execute the perceptron algorithm in a slightly different way. In the previous section, the perceptron algorithm visits each feature vector $\mathbf{x}_i$ and its label y_i in the training set S_{training} in a deterministic order (Line 5 for each feature ...). Now, we want to random sample next $\mathbf{x}_i$ and its label y_i from the training set S_{training} with replacement. The new algorithm is shown below:

Algorithm 2 Perceptron Learning Algorithm

Data: Training set S_{training}, which contains feature vectors X and labels Y;
Initialize parameters $\mathbf{w}$ and b; pick a constant $\lambda \in (0, 1]$, which is similar to the step size in the standard gradient descent algorithm ($\lambda = 1$ by default).
while *not every data point is correctly classified* **do**
  for *i in range(0, k)* **do**
    random sample each feature vector $\mathbf{x}_i$ and its label y_i from the training set S_{training}
    compute the model prediction $f(\mathbf{x}_i; \mathbf{w}, b)$;
    if $y_i = f(\mathbf{x}_i; \mathbf{w}, b)$ **then**
      continue;
    else
      update the parameters $\mathbf{w}$ and $\mathbf{b}$:
      $\mathbf{w} \Leftarrow \mathbf{w} + \lambda(y_i - f(\mathbf{x}_i; \mathbf{w}, b))\mathbf{x}_i$
      $\mathbf{b} \Leftarrow \mathbf{b} + \lambda(y_i - f(\mathbf{x}_i; \mathbf{w}, b))$
    end
  end
end

Please set $k = \text{len(training data)}$ and follow the instructions in the skeleton code and report:

1. Your code.
2. Run your code 3 times and draw 3 training error curves and the curve with deterministic training (Section 3.1). Compare and write your observation between the two sampling methods.

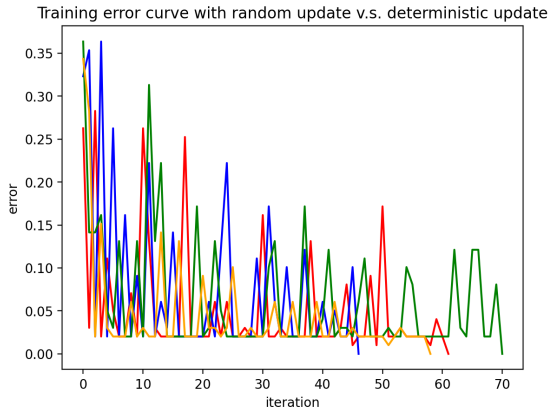
Hint: You can randomly sample the indices of training data from 0 to $k - 1$ and then use the index to get the corresponding sample each time. `random.choices(pop,n)` would return a n-sized list of elements chosen from the input list `pop` with replacement.

```
import random
import matplotlib.pyplot as plt

# Some settings.
errors_random_list = []           # All Error history (3 runs)
lam    = 1            # Lambda which controls the step size.

for _ in range(3):
    # Initialization.
    W      = np.zeros(2)  # Weight.
    b      = 0.0          # Bias.
    k      = len(X_train)
    # Perceptron learning algorithm.
    errors_random = [] # Error history for the current run
    while calc_error(X_train, Y_train, W, b) > 0:
        idxs = np.arange(k)
        np.random.shuffle(idxs)
        for i in range(k):
            idx = idxs[i]
            xi, yi = X_train[idx], Y_train[idx]
            y_pred = f_perceptron(xi, W, b)
            if yi == y_pred:
                continue
            else:
                W += lam * (yi - y_pred) * xi
                b += lam * (yi - y_pred)
        errors_random.append(calc_error(X_train, Y_train, W, b))
    errors_random_list.append(errors_random)
```

Solution:



Clearly, the results of the random sampling method are more volatile than the deterministic method. The deterministic method converged in about 60 iterations, while the random sampling method sometimes took less than 50 iterations and sometimes took more than 70 iterations. This indicates that out of all orders to visit the data points, the deterministic method was quite average.

Problem 4. (16 points) Circuit Diagram I

The computation of the backpropagation can be nicely visualized with a circuit diagram. Let's consider a complicated expression that involves multiple composite functions, and the function is defined as:

$$f(x, y, z) = (x + y)z \tag{4}$$

We visualize the function with a circuit diagram, which helps us intuitively understand the backpropagation. You can review it from *Fei-Fei Li, Andrej Karpathy*, and *Justin Johnson*'s Stanford CS231n course. Link: http://cs231n.github.io/optimization-2/ The visualization is shown in Fig. 2.

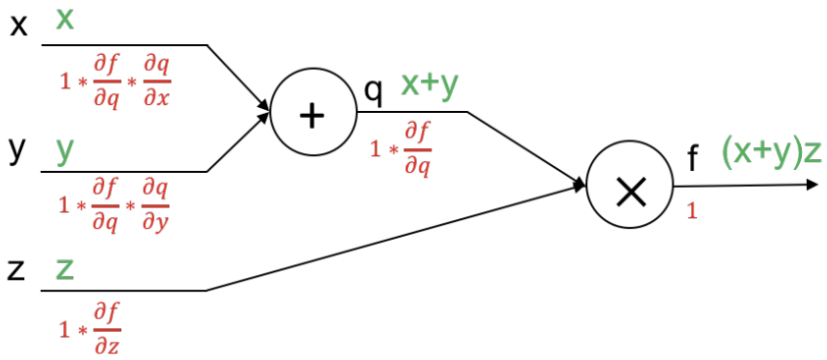


Figure 2: Circuit Diagram of the function $f(x, y, z) = (x + y)z$.

Particularly, this expression can be broken down into two expressions: $q = x + y$ and $f = qz$. According to **Chain Rule**, the derivative/gradient of f with respect to its inputs x, y, and z can be calculated easily with the intermediate variable q. Although we don't really care about the gradient of f with respect to q, but the gradient $\frac{\partial f}{\partial q}$ simplifies the calculation of the gradient.

Let's take a look at the circuit diagram. The letters in black represent the variables, which are input variables x, y, z, the intermediate variable q, and output f. The expressions in green represent the current values of the variables respectively, and the expressions in red represent the gradient of f with respect to the particular variables.

The circuit diagram visualizes how the gradient of f with respect to each variable is calculated. Let's follow the circuit diagram to perform a forward calculation and a backward calculation by hand.

In Fig. 3, suppose we have three inputs, which are $x = -2$, $y = 5$, and $z = -4$, and the circuit diagram represents the function $f(x, y, z) = (x + y)z$. We follow the steps below to calculate the value and the gradient with respect to each variable in the diagram.

1. $q = x + y = 3$
2. $f = qz = -12$
3. $\frac{\partial f}{\partial q} = \frac{\partial (qz)}{\partial q} = z = -4$
4. $\frac{\partial f}{\partial z} = \frac{\partial (qz)}{\partial z} = q = 3$
5. $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x} = -4 \cdot 1 = -4$
6. $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y} = -4 \cdot 1 = -4$

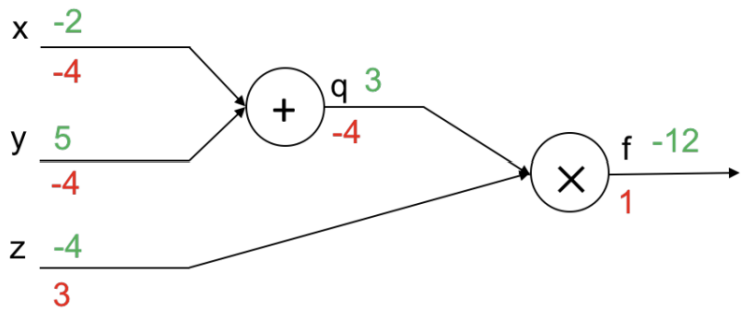


Figure 3: The real-valued circuit diagram of the function $f(x, y, z) = (x + y)z$.

Your tasks:

The circuit diagram for the function defined below is provided in Fig. 4:

$$f(x, y, z, w) = \ln[\max(x, y) \times (z + w)], \tag{5}$$

where $\max(x, y)$ takes the maximum value of x and y as output.

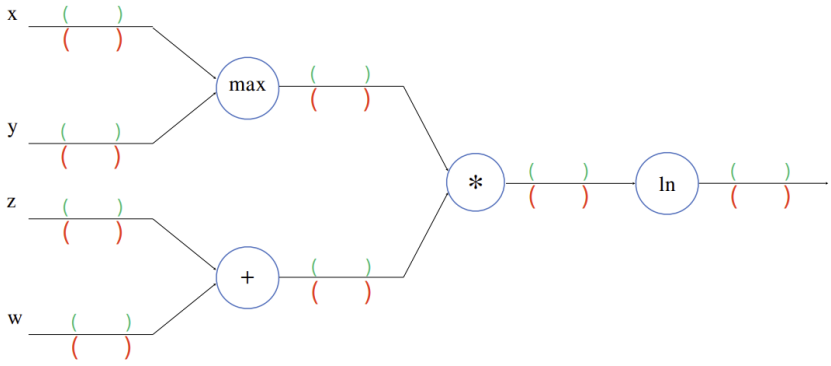
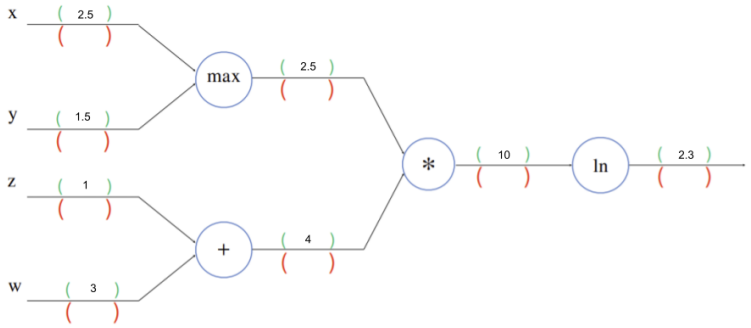


Figure 4: Circuit Diagram of the function $f(x, y, z, w)$. We use $*$ to denote the multiplication of the two scalar values.

Suppose the value for each input is $x = 2.5$, $y = 1.5$, $z = 1$ and $w = 3$.

4.1 (8 points) Perform the forward computation of the function f, and fill-in the circuit diagram in Fig. 4 with your calculated values.

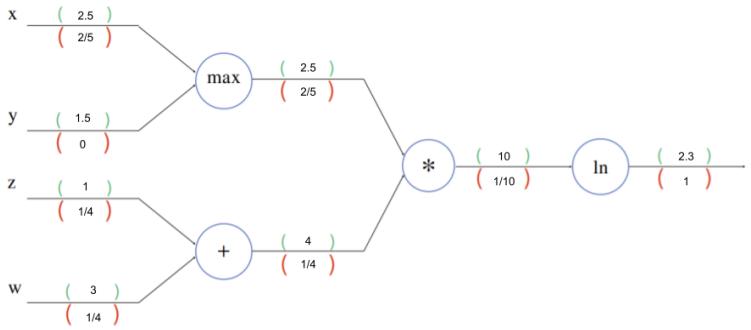
Solution:



$q = \max(x, y) = 2.5$, $p = z + w = 4$, $r = qp = 10$, $f = \ln(r) = \ln(10) \approx 2.3$

4.2 (8 points) Perform the backward computation of the function f, and fill-in the circuit diagram in Fig. 4 with your calculated gradients.

Solution:



$$\frac{\partial f}{\partial r} = \frac{\partial \ln(r)}{\partial r} = \frac{1}{r} = \frac{1}{10},$$
$$\frac{\partial f}{\partial q} = \frac{\partial f}{\partial r} \cdot \frac{\partial r}{\partial q} = \frac{1}{10} \cdot p = \frac{2}{5},$$
$$\frac{\partial f}{\partial p} = \frac{\partial f}{\partial r} \cdot \frac{\partial r}{\partial p} = \frac{1}{10} \cdot q = \frac{1}{4},$$
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x} = \frac{2}{5} \cdot \underbrace{int(x > y)}_{1} = \frac{2}{5},$$
$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial y} = \frac{2}{5} \cdot \underbrace{int(y > x)}_{0} = 0,$$
$$\frac{\partial f}{\partial z} = \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial z} = \frac{1}{4} \cdot 1 = \frac{1}{4},$$
$$\frac{\partial f}{\partial w} = \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial w} = \frac{1}{4} \cdot 1 = \frac{1}{4}$$

Problem 5. (16 points) Logistic Regression

Assume in a binary classification problem, we need to predict a binary label $y \in \{-1, +1\}$ for a feature vector $\mathbf{x} = [x_0, x_1]^\top$. In logistic regression, we can reformulate the binary classification problem in a probabilistic framework: We aim to model the distribution of classes given the input feature vector $\mathbf{x}$. Specifically, we can express the conditional probability $p(y|\mathbf{x})$ parameterized by $(\mathbf{w}, b)$ using a logistic function. Assume the probability of the positive prediction $p(y = +1|\mathbf{x})$ is represented as:

$$p(y = +1|\mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \tag{6}$$

5.1 (8 points) Basic Formulation

1. Please derive the formulation of $p(y = -1|\mathbf{x})$.
2. Please show that $p(y|\mathbf{x}) = \frac{1}{1+e^{-y(\mathbf{w}^\top \mathbf{x}+b)}}$.

Solution: Using the property of the sigmoid function, we have:

1. $p(y = -1|\mathbf{x}) = 1 - p(y = +1|\mathbf{x}) = 1 - \sigma(\mathbf{w}^\top \mathbf{x} + b) = \sigma(-\mathbf{w}^\top \mathbf{x} - b) = \frac{1}{1+e^{\mathbf{w}^\top \mathbf{x}+b}}$
2. $p(y|\mathbf{x}) = \sigma(y(\mathbf{w}^\top \mathbf{x} + b)) = \frac{1}{1+e^{-y(\mathbf{w}^\top \mathbf{x}+b)}}$

We can see that if $y = +1$, then $p(y = +1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$. If $y = -1$, then $p(y = -1|\mathbf{x}) = \sigma(-\mathbf{w}^\top \mathbf{x} - b)$ which matches our derivation.

5.2 (8 points) Derive Gradient

Given a training dataset $S_{\text{training}} = \{(\mathbf{x}_i, y_i), i = 1, \ldots, n\}$, we wish to optimize the negative log-likelihood loss $L(\mathbf{w}, b)$ of the logistic regression model defined above:

$$L(\mathbf{w}, b) = -\sum_{i=1}^{n} \ln p(y_i|\mathbf{x}_i) \tag{7}$$

where $p_i = p(y_i|\mathbf{x}_i)$. The optimal weight vector $\mathbf{w}$ and bias b are used to build the logistic regression model:

$$\mathbf{w}^*, b^* = \arg\min_{w,b} L(w, b) \tag{8}$$

In this problem, we attempt to obtain the optimal parameters $\mathbf{w}^*$ and b^* by using a standard gradient descent algorithm.

(a) Please show that $\frac{\partial L(\mathbf{w},b)}{\partial \mathbf{w}} = -\sum_{i=1}^{n}(1 - p_i)y_i\mathbf{x}_i$.

(b) Please show that $\frac{\partial L(\mathbf{w},b)}{\partial b} = -\sum_{i=1}^{n}(1 - p_i)y_i$.

Solution: Here we use the fact that the derivative of the sigmoid function is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, so let $z_i = y_i(\mathbf{w}^\top \mathbf{x}_i + b)$ hence

$$\frac{\partial p_i}{\partial z_i} = \sigma(z_i)(1 - \sigma(z_i)) = p_i(1 - p_i)$$

Now, using the chain rule, we have

$$\frac{\partial \ln p_i}{\partial z_i} = \frac{\partial \ln p_i}{\partial p_i} \cdot \frac{\partial p_i}{\partial z_i} = \frac{1}{p_i} p_i(1 - p_i) = 1 - p_i$$

Finally, we use chain rule again to find

$$\frac{\partial L}{\partial \mathbf{w}} = -\sum_{i=1}^{n} \frac{\partial \ln p_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial \mathbf{w}} = -\sum_{i=1}^{n}(1 - p_i)y_i\mathbf{x}_i$$

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{n} \frac{\partial \ln p_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial b} = -\sum_{i=1}^{n}(1 - p_i)y_i$$

Problem 6. (10 points) Multi-Layer Neural Networks

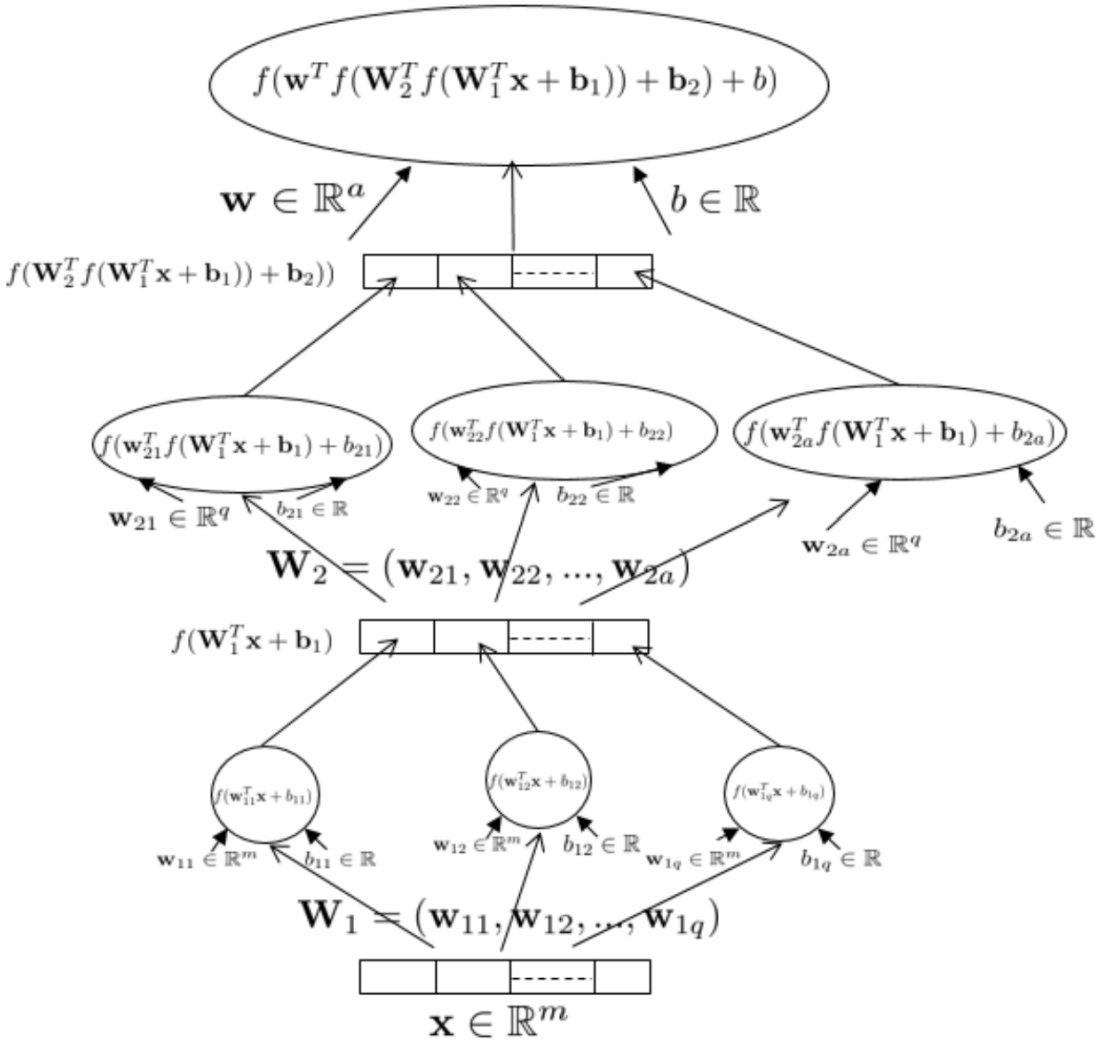


Figure 5: A multi-layer feedforward neural networks structure.

Figure 5 shows a multi-layer feedforward neural network structure, where $m = 5$, $q = 8$, and $a = 6$. Note that function $f(\mathbf{W})$ performs element-wise operations on an input matrix $\mathbf{W}$.

Please provide the sizes of the variables listed below.

1. x:
2. $\mathbf{W}_1$:
3. b_1:
4. $f(W_1^\top x + b_1)$:
5. $\mathbf{W}_2$:
6. b_2:
7. $f(\mathbf{W}_2^\top f(\mathbf{W}_1^\top x + b_1) + b_2)$:
8. w:
9. b:
10. $f(w^\top f(\mathbf{W}_2^\top f(\mathbf{W}_1^\top x + b_1) + b_2) + b)$:

Solution:

1. $x \in \mathbb{R}^5$
2. $\mathbf{W}_1 \in \mathbb{R}^{5\times 8}$
3. $b_1 \in \mathbb{R}^8$
4. $f(\mathbf{W}_1^\top x + b_1) : \mathbb{R}^8 \to \mathbb{R}^8$
5. $\mathbf{W}_2 \in \mathbb{R}^{8\times 6}$
6. $b_2 \in \mathbb{R}^6$
7. $f(\mathbf{W}_2^\top f(\mathbf{W}_1^\top x + b_1) + b_2) : \mathbb{R}^6 \to \mathbb{R}^6$
8. $w \in \mathbb{R}^6$
9. $b \in \mathbb{R}$
10. $f(w^\top f(\mathbf{W}_2^\top f(\mathbf{W}_1^\top x + b_1) + b_2) + b) : \mathbb{R} \to \mathbb{R}$

Problem 7. (10 points) Number of Parameters

Consider a one-hidden-layer network with 3-dimensional input, 2-dimensional output, and 5 hidden units as shown in Figure 6.

7.1 (2 points) How many neurons (nodes) are there in a neural network? Do not count the input layer.

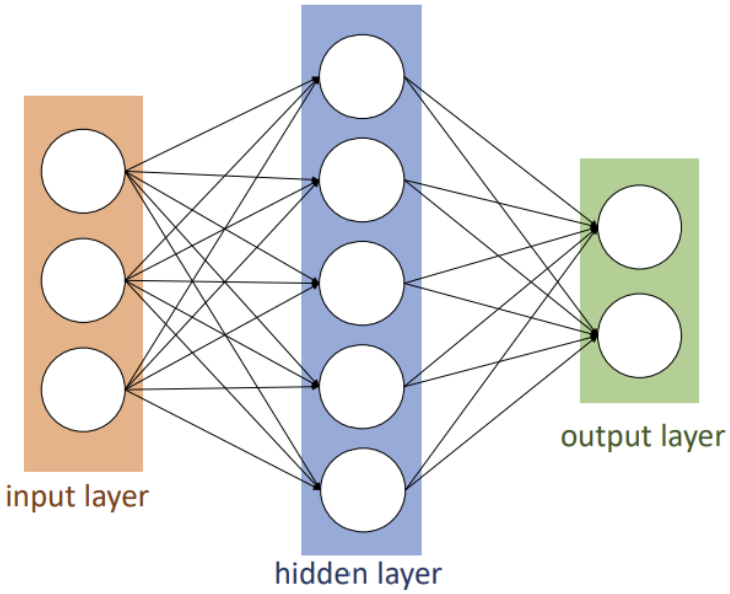


Figure 6: Question 7.

Solution: There are 5 in the hidden layer and 2 in the output layer which is 7 neurons in total.

7.2 (2 points) How many weights (edges) are there in the network?

Solution: There are 3×5 weights to the first layer and 5×2 weights to the second layer which is 25 weights in total.

7.3 (3 points) How many learnable parameters are there in total in the network? Include the bias parameters in your answer.

Solution: There are 25 weights and 7 neurons with biases which is 32 parameters in total.

7.4 (3 points) Now let's generalize the result from part 3. How many parameters are there in a one-hidden-layer network with D_x-dimensional input, D_y-dimensional output, and H hidden units? Include the bias parameters in your answer.

Solution: There are $D_x \times H$ weights to the first layer and $H \times D_y$ weights to the second layer which is $D_x \times H + H \times D_y + H + D_y$ parameters in total.